PROJECT AND TEAM INFORMATION

Project Title

(Try to choose a catchy title. Max 20 words).

Smart Hostel Management & Complaint Resolution System

Student / Team Information

Team Name: *Team #*	*Hostel- Nexus* *Team # DSCPP-III-2026-T269*
Team member 1 (Team Lead) *(Last Name, name: student ID: email, picture):*	Raj, Harshit - 2510011339 *rajharshit1807@gmail.com* 
Team member 2 *(Last Name, name: student ID: email, picture):*	Negi, Diviya - 2510010573 *diviyanegi51@gmail.com* 
Team member 3 *(Last Name, name: student ID: email, picture):*	Verma, Aditya – 2510380258 av0844769@gmail.com 

PROPOSAL DESCRIPTION (10 pts)

Motivation (1 pt)

(Describe the problem you want to solve and why it is important. Max 300 words).

In many college hostels, routine work is still handled with registers, notice boards and messages passed between students and the warden. Room and bed allotment is often written down manually at the beginning of a semester. This can create simple problems, such as a bed being shown as occupied in one record while another record still shows it as free. Complaints have a similar issue. A student may report a leaking tap, damaged switchboard or mess problem on paper or verbally, but there may be no clear record showing its current status or how long it has been waiting. Because of this, a small issue such as a noisy door can sometimes stay in the same queue as a serious electrical problem, even though the two should not have the same priority.

Visitor entries and leave or outpass requests also create extra paperwork. A warden may have to check several pages just to find out who is inside the hostel or which students are away. With more students, finding a particular room, checking previous occupancy, or looking up an old complaint takes even more time because the information is spread across separate records instead of being connected.

Our proposed system puts student, room, complaint, visitor and leave details in one place. It can also separate urgent complaints from normal ones and make common searches quicker. The main reason for choosing this problem is practical: hostel staff need a simple way to find the right record and notice important complaints without going through several registers. Faster handling of serious problems can improve both safety and everyday hostel life.

State of the Art / Current solution (1 pt)

(Describe how the problem is solved today (if it is). Max 200 words).

At present, many hostels use a combination of registers, spreadsheets and messages or notices for daily management. Room allocation is normally written down by the warden's office, while vacancy details may only be changed when staff update the record. This can result in a room or bed being assigned twice, or a free bed not being noticed. Complaints are usually entered in a register or told directly to the responsible person. They are generally dealt with in the order they are received, so there is no reliable method for putting a serious problem ahead of a routine one. Spreadsheets can help with student lists, but they do not always connect student, room, complaint, visitor and leave information. Therefore, finding one old record can still mean checking several files. Our project is a college-level prototype that uses DSA and C++ concepts to show how these records can be organized and handled more efficiently.

Project Goals and Milestones (2 pts)

(Describe the project general goals. Include initial milestones as well any other milestones. Max 300 words).

General Goals:

- □eep student details, room/bed data, complaints, visitors and leave/outpass entries together in one system instead of separate registers.
- Show room and bed availability clearly so that the same bed is not allotted twice.
- Store maintenance complaints and give urgent problems higher priority than routine complaints.
- Make it easier to find student, room and complaint records even when the number of entries increases.
- □eep proper entry/exit details for visitors and records of student leave or outpass.
- □eep previous room occupancy and complaint details so they can be checked later.
- □se DSA in C and object-oriented C++ together in one working hostel management project.

Milestones:

- Discuss the requirements and finalize the features and information that the system will store.
- Plan the main C/C++ data structures and classes such as Student, Room, Complaint, Visitor and Warden.
- Build student registration and room/bed allocation, using hash-based lookup for quick access.
- Handle complaints with a priority queue for urgent cases and a normal queue for routine cases.
- Add visitor entry/exit and leave/outpass records and connect them with student information.
- Add searching and sorting by room, urgency or date, and use linked lists for keeping history.
- Add file handling so saved records can be loaded again when the program starts.
- Combine the modules into one console program, test it with sample records, and check cases such as a full hostel, duplicate ID or invalid room number.

Project Approach (3 pts)

(Describe how you plan to articulate and design a solution. Including platforms and technologies that you will use. Max 300 words).

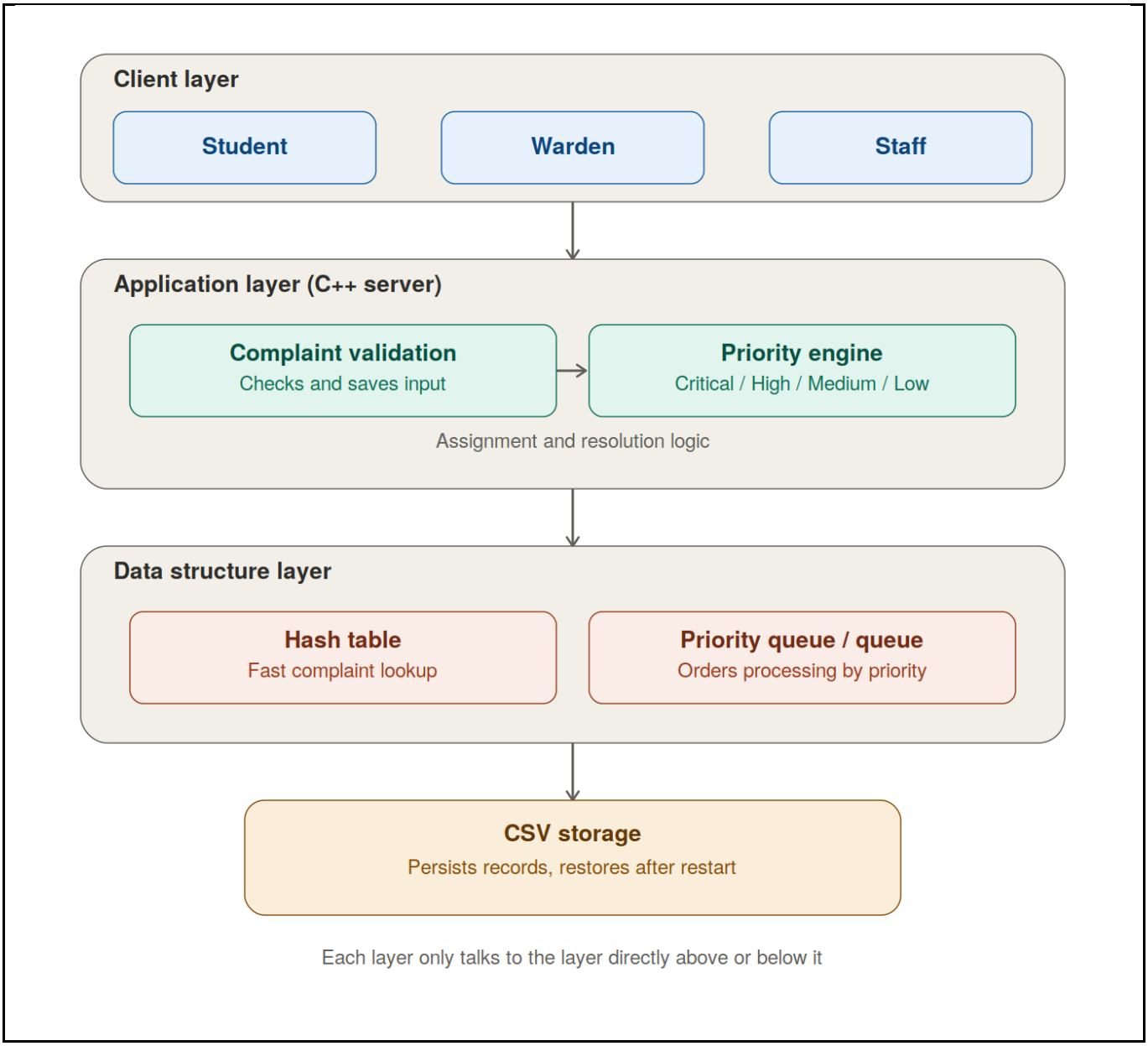
- We plan to make the project as a modular console application. The main DSA structures and algorithms will be written in C, while C++ will be used for classes and the overall application logic.
- For student, room and complaint records, we will use a hash table with the student ID or room number as the key. This should make common lookups faster than checking every record one by one. Complaints will be divided into urgent and routine cases. □rgent problems, for example an electrical fault or water leakage, will go into a priority queue so that more serious cases are taken first. Normal complaints will use a regular queue and follow the order in which they were received. Linked lists will keep room occupancy history and previous complaints because these records can keep growing. We will also use sorting algorithms when we need to arrange records by date, priority or other required fields.

Sorting will help arrange complaints by date or priority and list students or rooms in the required order. If a record cannot be found through the hash table, a suitable searching method will be used.

For the C++ part, classes will represent Student, Room, Complaint, Visitor and Warden. A Person base class will be used for Student and Warden to show inheritance and a virtual display function. Getters and setters will keep class data controlled. Constructors will set up objects, and exception handling will be used for inputs such as duplicate student IDs or invalid room numbers. Files will be used to save data between program runs. STL containers may be used where useful, but the main DSA structures required for the project will be implemented by us.

System Architecture (High Level Diagram)(2 pts)

(Provide an overview of the system, identifying its main components and interfaces in the form of a diagram using a tool of your choice).



Project Outcome / Deliverables (1 pts)

(Describe what are the outcomes / deliverables of the project. Max 200 words).

- Our final output will be a working C++ console prototype of the Smart Hostel Management System. It will demonstrate these main functions:
- Student registration and profile handling, with quick lookup using student ID or name.
- Room and bed allotment with vacancy updates so an already allotted bed cannot be assigned again.
- Complaint and maintenance entry, with urgent cases placed ahead of routine cases using a priority queue and

- Visitor entry and exit records, along with leave/outpass details connected to student records.
- Searching and sorting of student, room and complaint information using fields such as date or urgency.

  File storage so saved records are available after the program is restarted.
- Room occupancy history and complaint history maintained with linked lists.
- A test run with sample data, including cases such as a full hostel, an invalid room number and a duplicate student ID.
- The prototype will show the use of C for the main data structures and algorithms and C++ for the object-oriented part of the application, as planned in the project approach.

Assumptions

(Describe the assumptions (if any) you are making to solve the problem. Max 100 words)

- For this project, we assume one hostel, or a small fixed number of blocks, with a known number of rooms and beds. The student, complaint and visitor entries will be sample data entered during the demonstration and will not come from a live college database. The prototype will not use a live online database, biometric devices, GPS tracking or a payment system. It is meant to demonstrate DSA and C++ concepts rather than act as a complete hostel product. Complaint urgency will also be entered manually.

References

(Provide a list of resources or references you utilised for the completion of this deliverable. You may provide links).

- cppreference.com – C++ language and standard library reference (classes, inheritance, polymorphism, exception handling, STL): https://en.cppreference.com/
- cppreference.com – C language reference (arrays, pointers, structures, file handling): https://en.cppreference.com/w/c
- GeeksforGeeks – Data Structures and Algorithms: Hash Table, Priority Queue, Queue, Linked List, Searching and Sorting algorithms: https://www.geeksforgeeks.org/data-structures/
- GeeksforGeeks – File Handling in C/C++: https://www.geeksforgeeks.org/file-handling-c-classes/
- Course textbook/lecture notes on DSA (C) and Object-Oriented Programming (C++) – to be verified against the specific course reference list.